ORIENTATION TO COMPUTING

UNIT 2: NUMBER SYSTEMS, DATA REPRESENTATION & DIGITAL LOGIC

Curriculum: Lovely Professional University | School of Computer Science & Engineering
Reference: Data Communications and Networking by Behrouz A. Forouzan & Digital Design by M. Morris Mano

>> 2.1 Positional Number Systems and Radix Conversion

Computers represent all instructions and data using positional number systems based on discrete integer bases (radix r):
* Decimal System (Radix 10): Digits 0 through 9.
* Binary System (Radix 2): Digits 0 and 1. Fundamental language of electronic digital logic.
* Octal System (Radix 8): Digits 0 through 7 (Each octal digit maps to exactly 3 binary bits: 2^3 = 8).
* Hexadecimal System (Radix 16): Digits 0 through 9 and letters A (10) through F (15). (Each hex digit maps to exactly 4 binary bits / 1 nibble: 2^4 = 16).
* Conversion Principles:
- Radix r to Decimal: Polynomial expansion: Value = Sum(d_i x r^i) for integer and fractional components.
- Decimal to Radix r: Successive division by r for integer part (collect remainders in reverse order); successive multiplication by r for fractional part (collect integer overflows in forward order).

>> 2.2 Binary Integer Representation and Arithmetic

* 1. Unsigned Integers: All n bits represent magnitude. Range for n bits: [0 to 2^n - 1].
* 2. Sign-Magnitude: Most Significant Bit (MSB) is sign (0 = positive, 1 = negative); remaining n-1 bits represent magnitude. Drawbacks: Two representations of zero (+0 and -0); complex arithmetic hardware.
* 3. 1''s Complement: Negative number formed by bitwise inverting all bits (NOT operation). Still suffers from dual zero representation.
* 4. 2''s Complement (Universal Hardware Standard):
- Formulation: Negative of X is formed by taking 1''s complement of X and adding 1: 2''s Comp(X) = (~X) + 1.

- Advantages: Unique representation of zero (00000000); hardware subtraction is achieved by standard addition (A - B = A + (~B + 1)).
- Range for n bits: [-2^(n-1) to +2^(n-1) - 1]. For 8 bits: -128 to +127.
- Overflow Rule: When adding two numbers of the same sign, if the result produces an opposite sign bit, arithmetic overflow has occurred (Carry into MSB XOR Carry out of MSB = 1).

>> 2.3 Floating-Point Representation: IEEE 754 Standard

Real numbers with fractional parts and wide dynamic ranges are represented using normalized scientific notation:
* Single Precision (32-bit):
- 1 Sign bit (S): 0 for positive, 1 for negative.
- 8 Exponent bits (E): Biased exponent using Bias = 127. Actual exponent e = E - 127.
- 23 Mantissa / Fraction bits (M): Normalized with an implicit leading 1: Significant = 1.M.
- Value = (-1)^S x (1.M) x 2^(E - 127).
* Double Precision (64-bit): 1 Sign bit, 11 Exponent bits (Bias = 1023), 52 Mantissa bits.

>> 2.4 Character and Symbol Encoding Systems

* ASCII (American Standard Code for Information Interchange): 7-bit encoding representing 128 characters (0-127). Extended ASCII uses 8 bits for 256 symbols.
* Unicode Standard: Universal character set assigning unique code points (U+XXXX) to all world languages and symbols:
- UTF-8: Variable-length encoding (1 to 4 bytes per character). 100% backward compatible with ASCII (1 byte). Dominates the World Wide Web (>98% of web pages).
- UTF-16: 2 or 4 bytes per character; standard internal encoding in Java and Windows OS.

>> 2.5 Digital Logic Gates and Boolean Minimization

* Fundamental Logic Gates: AND (F = A.B), OR (F = A + B), NOT (F = ~A).
* Universal Logic Gates: NAND (~(A.B)) and NOR (~(A + B)). Any digital logic circuit can be realized exclusively using only NAND or only NOR gates.
* Exclusive Gates: XOR (F = A (+) B = A''B + AB'', odd parity detector) and XNOR (F = AB + A''B'', equivalence comparator).
* Boolean Algebra Axioms & De Morgan''s Laws:
- Law 1: ~(A + B) = ~A . ~B (NOT of OR is AND of NOTs).

- Law 2: ~(A . B) = ~A + ~B (NOT of AND is OR of NOTs).
* Karnaugh Maps (K-Maps): Graphical minimization technique utilizing Gray code ordering where adjacent cells differ by exactly one binary bit, enabling systematic identification of prime implicants.

[PROBLEM 1: Multi-Base Conversion with Fractional Component]

Problem Statement: Convert the decimal number 218.625 into: (1) Binary, (2) Octal, and (3) Hexadecimal.
Step-by-Step Mathematical Derivation:
* 1. Integer Part Conversion (218):
- 218 / 2 = 109 rem 0 | 109 / 2 = 54 rem 1 | 54 / 2 = 27 rem 0 | 27 / 2 = 13 rem 1 | 13 / 2 = 6 rem 1 | 6 / 2 = 3 rem 0 | 3 / 2 = 1 rem 1 | 1 / 2 = 0 rem 1.
- Reading remainders from bottom to top: 218_10 = 11011010_2.
* 2. Fractional Part Conversion (0.625):
- 0.625 x 2 = 1.250 -> Integer 1, fractional 0.250.
- 0.250 x 2 = 0.500 -> Integer 0, fractional 0.500.
- 0.500 x 2 = 1.000 -> Integer 1, fractional 0.000 (terminates).
- Reading integers top to bottom: 0.625_10 = 0.101_2.
* 3. Binary Result: 218.625_10 = 11011010.101_2.
* 4. Conversion to Octal (Group in 3s from binary point):
- Integer: (011) (011) (010) = 3 3 2.
- Fractional: (101) = 5.
- Octal Result: 332.5_8.
* 5. Conversion to Hexadecimal (Group in 4s from binary point):
- Integer: (1101) (1010) = D A.
- Fractional: (1010) = A.
- Hexadecimal Result: DA.A_16 or 0xDA.A.

[PROBLEM 2: 8-Bit 2''s Complement Subtraction and Overflow Detection]

Problem Statement: Perform the following operations using 8-bit 2''s complement arithmetic and verify whether arithmetic overflow occurs: (A) (+68) - (+35), and (B) (+85) + (+60).
Mathematical Execution:
* Part A: (+68) - (+35) = (+68) + (-35):
- +68 in binary = 01000100_2.
- +35 in binary = 00100011_2 -> 1''s comp = 11011100 -> 2''s comp (-35) = 11011101_2.

- Addition: 01000100 + 11011101 = (1) 00100001_2 (discard end-around carry).
- Result: 00100001_2 = 32 + 1 = +33 in decimal. Check: 68 - 35 = 33 (Correct!).
- Overflow check: Adding positive and negative numbers CANNOT cause overflow.
* Part B: (+85) + (+60):
- +85 in binary = 01010101_2.
- +60 in binary = 00111100_2.
- Addition: 01010101 + 00111100 = 10010001_2.
- Result Analysis: Both operands are POSITIVE (MSB = 0), but the sum has MSB = 1 (NEGATIVE!).
- True arithmetic sum = 145, which exceeds the maximum positive limit of 8-bit signed integer (+127).
- Conclusion: OVERFLOW OCCURRED! The CPU sets the Overflow Flag (V = 1).

[PROBLEM 3: IEEE 754 Single-Precision Floating-Point Encoding]

Problem Statement: Encode the decimal number -43.75 into 32-bit IEEE 754 Single-Precision floating-point format. Express the final bit sequence in Hexadecimal.
Step-by-Step Derivation:
* 1. Determine Sign Bit (S): Number is negative -> S = 1.
* 2. Convert Absolute Value to Binary: 43.75:
- 43 = 32 + 8 + 2 + 1 = 101011_2. 0.75 = 0.5 + 0.25 = 0.11_2.
- 43.75_10 = 101011.11_2.
* 3. Normalize into Scientific Notation (1.M x 2^e):
- 101011.11 = 1.0101111 x 2^5 (Shifted binary point 5 positions to the left). Actual exponent e = 5.
* 4. Calculate Biased Exponent (E):
- E = e + Bias = 5 + 127 = 132.
- 132 in 8-bit binary = 128 + 4 = 10000100_2.
* 5. Determine 23-bit Mantissa (M):
- Drop leading 1: M = 01011110000000000000000 (padded with trailing zeros to 23 bits).
* 6. Assemble 32-bit Pattern:
- S (1 bit): 1
- E (8 bits): 10000100
- M (23 bits): 01011110000000000000000
- Concatenated: 1 10000100 01011110000000000000000
* 7. Hexadecimal Representation (Group into nibbles):
- 1100 0010 0010 1111 0000 0000 0000 0000 -> 0xC22F0000.

[PROBLEM 4: 4-Variable K-Map Minimization and NAND Circuit Implementation]

Problem Statement: Minimize the Boolean switching function F(A, B, C, D) = Sum m(0, 2, 5, 7, 8, 10, 13, 15) using a 4-variable Karnaugh Map, and draw the equivalent circuit using only universal 2-input NAND gates.

K-Map Minimization Protocol:

* 1. K-Map Cell Population (Gray Code grid AB x CD):
- Min-terms at (0, 2), (8, 10) form a 4-corner quad: m0 (0000), m2 (0010), m8 (1000), m10 (1010).
- Common variables across the 4 corners: B is 0 (B''), D is 0 (D''). Term 1 = B''.D''.
- Min-terms at (5, 7, 13, 15) form an interior quad: m5 (0101), m7 (0111), m13 (1101), m15 (1111).
- Common variables across the interior quad: B is 1 (B), D is 1 (D). Term 2 = B.D.
* 2. Minimized Sum of Products (SOP): F = B''.D'' + B.D (Notice this is the Boolean equivalence XNOR: F = ~(B (+) D)).
* 3. Universal NAND Gate Realization:
- Apply double negation: F = ~~F = ~(~(B''.D'') . ~(B.D)).
- Invert B and D using NAND gates with tied inputs to produce B'' and D''.
- Feed B'' and D'' into NAND gate 1 to produce ~(B''.D'').
- Feed B and D into NAND gate 2 to produce ~(B.D).
- Feed the outputs of NAND 1 and NAND 2 into output NAND gate 3 to yield F.